

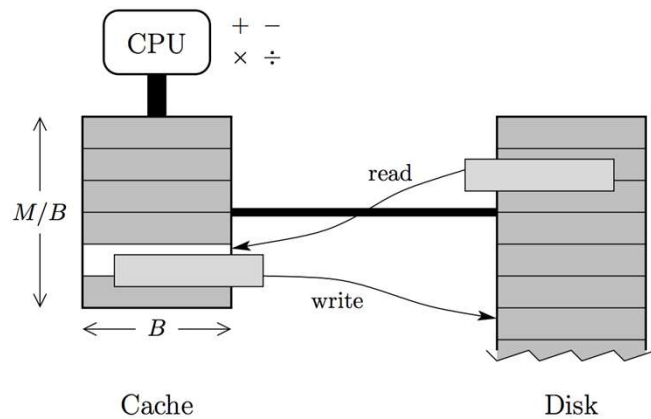
IO-Efficient Algorithms

IO-Efficient Algorithms

- External Memory Algorithms
- Cache-efficient Algorithms
- Cache-oblivious Algorithms

External memory

- Execution time governed by memory (data) transfers.



re 1. A two-level memory hierarchy shown with one block replacing another.

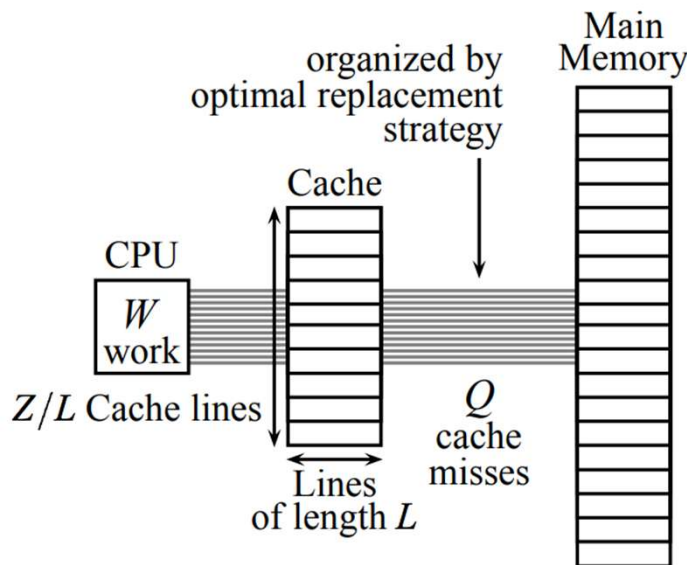
[1] The input/output complexity of sorting and related problems.

Merge sort 1 TB data

- Memory limitation
3 GB memory

Cache oblivious model

- Design cache-efficient algorithms without knowing Z and L .
- The ideal Cache Model:



The cache is partitioned into cache lines, each consisting of L consecutive words which are always moved together between cache and main memory.

Ideal Cache Model

- The processor can only reference words that reside in the cache.
 - If the referenced word belongs to a line already in cache, a **cache hit** occurs, and the word is delivered to the processor.
 - Otherwise, a **cache miss** occurs, and the line is fetched into the cache.
 - If the cache is full, a cache line must be **evicted**.
-
- The ideal cache uses the optimal off-line strategy of replacing the cache line whose next access is furthest in the future.
 - In reality, LRU – Least recently used.

Cache Eviction

- Least Recently Used
- Optimal Replacement
Farthest in future (Belady's algo.)

Offline vs Online

Greedy algo.

Cache eviction algorithms

Lemma 1 ([FLPR99, Lemma 12]). *If an algorithm makes T memory transfers on a cache of size $M/2$ with optimal replacement, then it makes at most $2T$ memory transfers on a cache of size M with LRU or FIFO replacement (and the same block size B).*

In other words, LRU and FIFO replacement do just as well as optimal replacement up to a constant factor of memory transfers and up a constant factor wastage of the cache. This competitiveness property of LRU and FIFO goes back

Add, scan, max.

Theorem 1. *Scanning N elements stored in a contiguous segment of memory costs at most $\lceil N/B \rceil + 1$ memory transfers.*

Practical Scanning

- Does factor of B matter?
- Should I presort my linked list before traversal?

Example.

- $N = 256,000,000 \sim 1\text{GB}$.
- $B = 8000 \sim 32\text{KB}$.
- 1 ms disk access time.

- N memory transfers take 256,000 sec. ~ 71 hours.
- N/B memory transfers take $256/8 = 32$ seconds

Reversing an array

- Data movement complexity?

Binary Search

$$T(N) = T(N/2) + O(1).$$

In the base case, $T(O(B)) = O(1)$

Theorem 3. *Binary search on a sorted array incurs $\Theta(\lg N - \lg B)$ memory transfers.*

However, in external memory,
searching in a sorted list can be done in as few as $\Theta(\lg N / \lg B) = \Theta(\lg_B N)$
memory transfers using B-trees.

B-tree

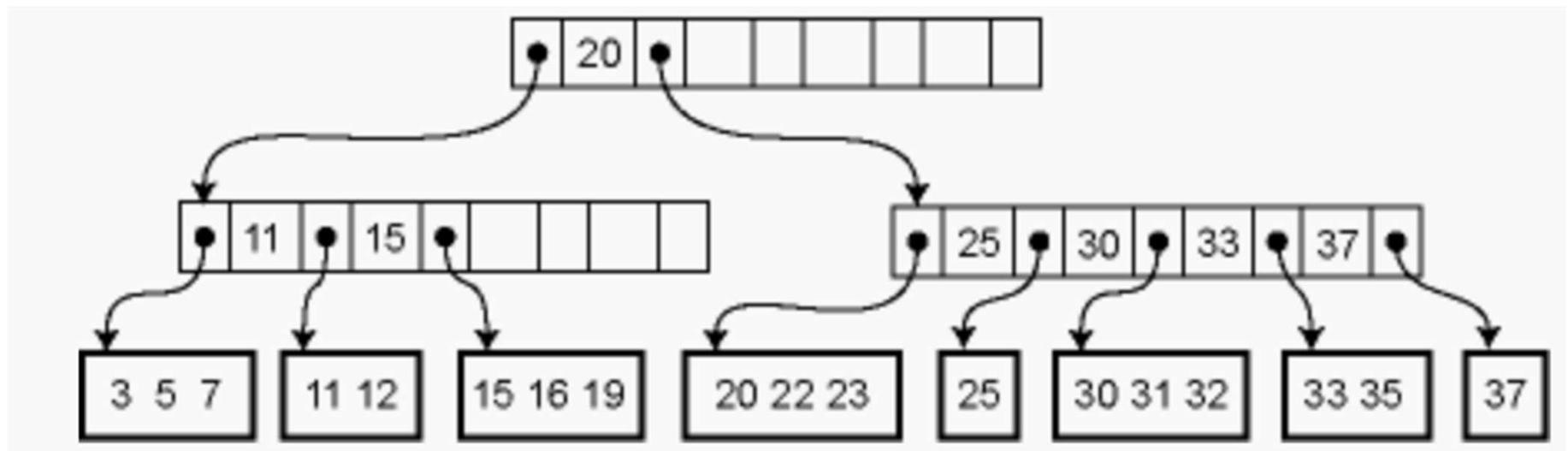
Input = N items

Each node contains $O(B)$ items.

Tree height = $O(\log_B N) = O(\log N / \log B)$

Number of blocks transferred for search operation = Tree Height

This is optimal.



External memory search

- Binary Search $O(\log N - \log B)$
- B-tree Search $O(\log N / \log B)$

Example.

- $N = 256,000,000,000$.
- $B = 8192$.
- Binary Search $O(\log N - \log B) = 38 - 13 \sim 25$
- B-tree Search $O(\log N / \log B) = 38/13 \sim 3$

Merge

- Binary merge
- N-ary merge

Merge lists in parallel

Visiting N elements in order costs $O(1 + N/B)$ memory transfers

We can run $\leq M/B$ parallel scans, keeping 1 block per scan in cache.

E.g., merge $O(M/B)$ lists of total size N in $O(1+N/B)$ memory transfers

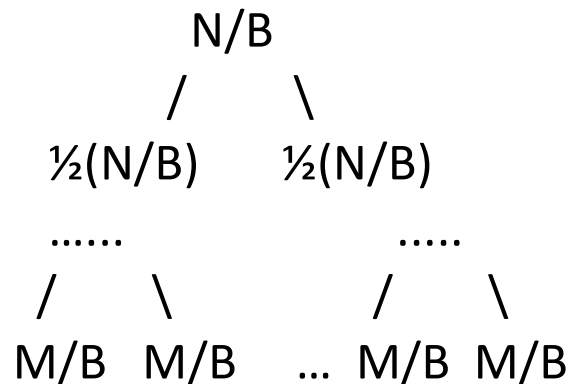
Mergesort

Memory Transfer recurrence

$$MT(N) = 2 MT(N/2) + O(N/B)$$

Base case: $MT(M) = O(M/B)$

Recurrence diagram



$$\text{Height} = \lg(N/B) - \lg(M/B) = \lg_2(N/M)$$

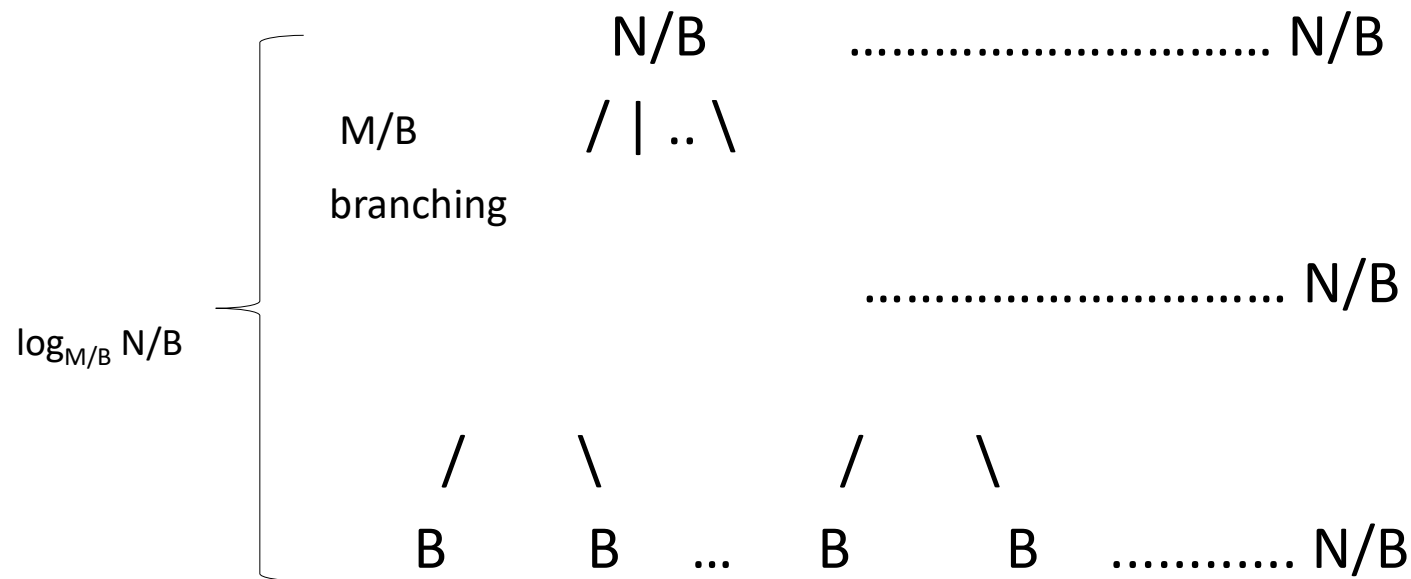
$$\text{Per level merge cost} = N/B$$

$$MT(N) = O(N/B \lg_2(N/M))$$

External Memory mergesort

- M/B way mergesort: During the merge, each memory block maintains the first B elements of each list, and when a block empties, the next block from that list is loaded.
- Merge effectively corresponds to scanning through the entire data, for a cost of $\Theta(N/B)$ memory transfers.

$O(\frac{M}{B})$ -way Mergesort



$$T(N) = \frac{M}{B} T\left(N/\frac{M}{B}\right) + O\left(\frac{N}{B}\right)$$

$$T(B) = O(1)$$

$$T(N) = O\left(\frac{N}{B} \log_{M/B} \frac{N}{B}\right)$$

External Memory mergesort

The total number of memory transfers for this sorting algorithm is given by the recurrence

$$T(N) = \frac{M}{B}T(N/\frac{M}{B}) + \Theta(N/B),$$

with a base case of $T(O(B)) = O(1)$. The recursion tree has $\Theta(N/B)$ leaves, for a leaf cost of $\Theta(N/B)$. The root node has divide-and-merge cost $\Theta(N/B)$ as well, as do all levels in between. The number of levels in the recursion tree is $\log_{M/B} N$, so the total cost is $\Theta(\frac{N}{B} \log_{M/B} \frac{N}{B})$.